

PART 1 — G#_constraints.sql

A. Add/Modify Constraints

You need at least some simple integrity constraints:

```
ALTER TABLE Resident
ADD CONSTRAINT chk_gender CHECK (Gender IN ('Male','Female'));
```

```
ALTER TABLE Resident
ADD CONSTRAINT chk_dob CHECK (DateOfBirth < SYSDATE);
```

B. Create an Audit Table

This is required.

```
CREATE TABLE Resident_Audit (
    audit_id    NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    resident_id NUMBER,
    old_history  VARCHAR2(500),
    new_history  VARCHAR2(500),
    changed_by   VARCHAR2(100),
    changed_on   DATE
);
```

C. Validation Trigger (BEFORE INSERT)

Example: prevent adding residents younger than 18.

```
CREATE OR REPLACE TRIGGER trg_resident_validate
BEFORE INSERT ON Resident
FOR EACH ROW
BEGIN
    IF MONTHS_BETWEEN(SYSDATE, :NEW.DateOfBirth)/12 < 18 THEN
        RAISE_APPLICATION_ERROR(-20020, 'Resident must be at least 18.');
```

D. Audit Trigger (AFTER UPDATE)

Audit medical history changes.

```
CREATE OR REPLACE TRIGGER trg_resident_audit
AFTER UPDATE OF MedicalHistory ON Resident
FOR EACH ROW
BEGIN
    INSERT INTO Resident_Audit
        (resident_id, old_history, new_history, changed_by, changed_on)
    VALUES
        (:OLD.ResidentID, :OLD.MedicalHistory, :NEW.MedicalHistory,
        USER, SYSDATE);
END;
/
```

★ PART 2 — G#_package.sql

Your uploaded file contains CRUD procedures.

We will **move them into a package** and add required extras.

A. Package Specification

(Minimum requirements)

```
CREATE OR REPLACE PACKAGE nursing_home_pkg AS

    -- CRUD (from your file)
    PROCEDURE add_resident(...);
    PROCEDURE get_resident_info(p_id IN NUMBER);
    PROCEDURE update_resident_history(p_id IN NUMBER, p_history IN VARCHAR2);
    PROCEDURE delete_resident(p_id IN NUMBER);

    -- Helper functions
    FUNCTION resident_exists(p_id NUMBER) RETURN BOOLEAN;

    -- Overloaded example
    PROCEDURE add_resident(p_id NUMBER, p_fname VARCHAR2, p_lname VARCHAR2);
```

```

-- Bulk operation
PROCEDURE bulk_update_rooms;

-- Ten project questions (examples)
PROCEDURE q1_residents_per_room_type;
PROCEDURE q2_residents_with_many_visitors;
PROCEDURE q3_oldest_residents;
PROCEDURE q4_male_female_count;
PROCEDURE q5_room_capacity_usage;
PROCEDURE q6_visitors_per_day;
PROCEDURE q7_resident_careplan_count;
PROCEDURE q8_room_with_most_residents;
PROCEDURE q9_resident_age_list;
PROCEDURE q10_history_keyword_search(keyword VARCHAR2);

END nursing_home_pkg;
/

```

B. Package Body

1. Include your CRUD code

Paste your procedures & functions here (already uploaded).

Example:

```

CREATE OR REPLACE PACKAGE BODY nursing_home_pkg AS

FUNCTION resident_exists(p_id NUMBER)
RETURN BOOLEAN AS
    v_count NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_count FROM Resident WHERE ResidentID = p_id;
    RETURN v_count > 0;
END;

```

(Continue adding your existing CRUD code here)

2. Overloaded Procedure

```

PROCEDURE add_resident(p_id NUMBER, p_fname VARCHAR2, p_lname VARCHAR2) AS
BEGIN

```

```
add_resident(p_id, NULL, p_fname, p_lname, TO_DATE('1950-01-01','YYYY-MM-DD'),  
            'Male', NULL);  
END;
```

3. Private Helper Procedure

(Useful for triggers/projects)

```
PROCEDURE log_msg(p_text VARCHAR2) IS  
BEGIN  
    DBMS_OUTPUT.PUT_LINE('[LOG] '||p_text);  
END;
```

4. Bulk Operation Requirement

Option A — BULK COLLECT + FORALL

Example: assign all residents to room 101.

```
PROCEDURE bulk_update_rooms AS  
    TYPE id_table IS TABLE OF NUMBER;  
    ids id_table;  
BEGIN  
    SELECT ResidentID BULK COLLECT INTO ids FROM Resident;  
  
    FORALL i IN 1..ids.COUNT  
        UPDATE Resident  
        SET RoomNumber = 101  
        WHERE ResidentID = ids(i);  
  
    DBMS_OUTPUT.PUT_LINE(ids.COUNT || ' residents reassigned.');
```

END;

5. Ten Project Question Procedures (examples)

```
PROCEDURE q1_residents_per_room_type AS  
BEGIN
```

```

DBMS_OUTPUT.PUT_LINE('--- Q1: Residents per Room Type ---');
FOR r IN (
    SELECT roomtype, COUNT(*) AS total
    FROM Resident JOIN Room USING(RoomNumber)
    GROUP BY roomtype
) LOOP
    DBMS_OUTPUT.PUT_LINE(r.roomtype || ': ' || r.total);
END LOOP;
END;

```

```

PROCEDURE q2_residents_with_many_visitors AS
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- Q2: Residents with >3 total visitors ---');
    FOR r IN (
        SELECT residentid, COUNT(*) AS total_visitors
        FROM Visitors
        GROUP BY residentid
        HAVING COUNT(*) > 3
    ) LOOP
        DBMS_OUTPUT.PUT_LINE('Resident ' || r.residentid || ' → ' || r.total_visitors || ' visitors');
    END LOOP;
END;

```

```

PROCEDURE q3_oldest_residents AS
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- Q3: Oldest 5 Residents ---');
    FOR r IN (
        SELECT residentid, firstname, lastname, dateofbirth
        FROM Resident
        ORDER BY dateofbirth
        FETCH FIRST 5 ROWS ONLY
    ) LOOP
        DBMS_OUTPUT.PUT_LINE(
            r.residentid || ' - ' || r.firstname || ' ' || r.lastname ||
            ' (DOB: ' || TO_CHAR(r.dateofbirth, 'YYYY-MM-DD') || ')'
        );
    END LOOP;
END;

```

```

PROCEDURE q4_male_female_count AS
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- Q4: Gender Distribution ---');
    FOR r IN (

```

```

        SELECT gender, COUNT(*) AS total
        FROM Resident
        GROUP BY gender
    ) LOOP
        DBMS_OUTPUT.PUT_LINE(r.gender||': '||r.total);
    END LOOP;
END;

```

```

PROCEDURE q5_room_capacity_usage AS
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- Q5: Room Occupancy vs Capacity ---');
    FOR r IN (
        SELECT room.roomnumber,
               room.capacity,
               COUNT(resident.residentid) AS occupants
        FROM room LEFT JOIN resident
        ON room.roomnumber = resident.roomnumber
        GROUP BY room.roomnumber, room.capacity
        ORDER BY room.roomnumber
    ) LOOP
        DBMS_OUTPUT.PUT_LINE(
            'Room '||r.roomnumber||
            ': '||r.occupants||'/'||r.capacity||' occupied'
        );
    END LOOP;
END;

```

```

PROCEDURE q6_visitors_per_day AS
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- Q6: Visitors per Day ---');
    FOR r IN (
        SELECT visitdate, COUNT(*) AS total
        FROM Visitors
        GROUP BY visitdate
        ORDER BY visitdate
    ) LOOP
        DBMS_OUTPUT.PUT_LINE(
            TO_CHAR(r.visitdate,'YYYY-MM-DD')||': '||r.total||' visitors'
        );
    END LOOP;
END;

```

```

PROCEDURE q7_resident_careplan_count AS
BEGIN

```

```

DBMS_OUTPUT.PUT_LINE('--- Q7: Care Plan Count Per Resident ---');
FOR r IN (
    SELECT residentid, COUNT(*) AS total_plans
    FROM CarePlan
    GROUP BY residentid
    ORDER BY total_plans DESC
) LOOP
    DBMS_OUTPUT.PUT_LINE(
        'Resident '||r.residentid||' → '||r.total_plans||' plans'
    );
END LOOP;
END;

```

```

PROCEDURE q8_room_with_most_residents AS
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- Q8: Room with Most Residents ---');
    FOR r IN (
        SELECT roomnumber, COUNT(*) AS total
        FROM Resident
        GROUP BY roomnumber
        ORDER BY total DESC
        FETCH FIRST 1 ROW ONLY
    ) LOOP
        DBMS_OUTPUT.PUT_LINE(
            'Room '||r.roomnumber||' has the most residents: '||r.total
        );
    END LOOP;
END;

```

```

PROCEDURE q9_resident_add_history_note AS
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- Q9: Add history note to all male residents (trigger fires) ---');

```

```

    UPDATE Resident
    SET MedicalHistory = NVL(MedicalHistory, '') || ' | Routine male health check'
    WHERE Gender = 'Male';

```

```

    DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' rows updated. ');
END;

```

```

PROCEDURE q10_history_keyword_search(keyword VARCHAR2) AS
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- Q10: Residents whose history contains keyword ---');

```

```

FOR r IN (
    SELECT residentid, firstname, lastname, medicalhistory
    FROM Resident
    WHERE LOWER(medicalhistory) LIKE '%' || LOWER(keyword) || '%'
) LOOP
    DBMS_OUTPUT.PUT_LINE(
        r.residentid||' - '||r.firstname||' '||r.lastname||
        ' | History: '||r.medicalhistory
    );
END LOOP;
END;

```

HOW TO TEST YOUR PACKAGE

In SQL Developer:

```

BEGIN
    nursing_home_pkg.add_resident(200,'Jack','Hill');
    nursing_home_pkg.get_resident_info(200);
    nursing_home_pkg.update_resident_history(200,'Test update');
    nursing_home_pkg.bulk_update_rooms;
    nursing_home_pkg.q1_residents_per_room_type;
END;
/

```